Java Programming Language Evaluation Report

I. Suitability for the Problem

A. Java is well suited to the Banking and Currency Exchange Application because it fits well into the object-oriented model and its standard library. In this implementation, Account class encapsulates account name, balance and currency, ExchangeRates class owns recorded rates and conversion logic and the main class only handles menu and front-end functionality.

B. All monetary values use java.mathBigDecimal with explicit RoundingMode which avoids binary floating point errors (1.0 + 2.0 = 3.000004) and gives exact centavo rounding.

Code Block 1. Monetary state and constants stored as BigDecimal in Account

```
public static final BigDecimal OPENING_BALANCE = new BigDecimal("1000.00"); //1000 peso base
public static final BigDecimal ANNUAL_INTEREST_RATE = new BigDecimal("0.05"); // 5% per annum
private static final BigDecimal DAYS_IN_YEAR = new BigDecimal("365");

private String name;
private BigDecimal balance;
private final ExchangeRates.Currency currency;
```

C. Java's static typing, enums, and generic collections also allow the six required currencies, account list, and interest table to be modeled directly.

Code Block 2. The Currency enum and an EnumMap holding the rate of each currency against PHP

```
public enum Currency
{
    PHP("Philippine Peso"),
    USD("United States Dollar"),
    JPY("Japanese Yen"),
    GBP("British Pound Sterling"),
    EUR("Euro"),
    CNY("Chinese Yuan Renminni");

    private final String fullName;

    Currency(String fullName)
    {
        this.fullName = fullName;
    }

    public String getLabel()
    {
        return fullName + " (" + name() + ")";
    }

    public static Currency fromMenuNumber(int number)
    {
        return values()[number - 1];
    }
}

public static final Currency BASE = Currency.PHP;
private static final int WORKING_SCALE = 10;
private final Map<Currency, BigDecimal> ratesToBase = new EnumMap<>(Currency.class);
```

D. The enum restricts currencies to the six values in REQ-0001 and REQ-0002, so an invalid currency cannot be constructed, while the EnumMap behaves like a hash map keyed by currency and is stored as a compact array internally. The same enum drives the numbered currency menus through ordinal() and values(), so the menu and the data cannot drift apart.

II. Expressiveness and Readability

The Java code in this project is straightforward and easy to follow. Tasks are broken into small helper methods (readInt, readPositiveDecimal, askYesNo, selectAccount), with clear names and modern features like arrow switches and enhanced for-loops to cut down repetitive code.

Code Block 3. Switch Expression of the main menu choice

```
switch (readInt("Choice: ", 1, 6))
{
    case 1 -> registerAccountScreen(accounts);
    case 2 -> depositScreen(accounts, rates);
    case 3 -> withdrawScreen(accounts, rates);
    case 4 -> currencyExchangeScreen(rates);
    case 5 -> recordExchangeRateScreen(rates);
    case 6 -> showInterestScreen(accounts);
}
```

The tradeoff is classic Java verbosity: without operator overloading, a simple daily interest formula turns into a long chain of method calls, and every field still requires its own getter.

Code Block 4. Daily interest formula written with complicated BigDecimal method calls

```
public static BigDecimal computeDailyInterest(BigDecimal endOfDayBalance)
{
    return endOfDayBalance.multiply(ANNUAL_INTEREST_RATE).divide(DAYS_IN_YEAR, 2, RoundingMode.HALF_UP);
}
```

III. Reliability and Safety

Java provides several mechanisms that make this program more dependable. Static typing catches mismatched types at compile time, exceptions separate error handling from normal flow, and automatic garbage collection removes manual memory management. In the implementation, Account.toValidMoney is the single point of validation for every deposit and withdrawal, it rejects null, rounds to two decimal places, and rejects amounts that are zero or negative after rounding.

withdraw() throws IllegalArgumentException when the amount exceeds the balance, the screens catch it and print a message, and the input helpers catch NumberFormatException and re-prompt. Using BigDecimal throughout also means balances never accumulate floating-point drift.

Code Block 5. Money validation and overdraft protection is centralized in Account

```java
private static BigDecimal toValidMoney(BigDecimal amount)
{
    if (amount == null)
    {
        throw new IllegalArgumentException("Amount is required.");
    }
    BigDecimal money = amount.setScale(2, RoundingMode.HALF_UP); // Standardizes amount to 2 decimal places (centavos)
    if (money.signum() <= 0)
    {
        throw new IllegalArgumentException("Amount must be greater than zero.");
    }
    return money;
}
```

IV. Tooling and Ecosystem

Java has a mature, standardized toolchain. The Java Development Kit (JDK) provides the compiler (javac) and the runtime (java); the project is built with javac *Account.java*, *ExchangeRates.java MP_11_Java.java* and started with java MP_11_Java. Because each public class must be stored in a file with the same name, the files must be named Account.java, ExchangeRates.java, and MP_11_Java.java, and a copy with a suffix such as Account_1.java will not compile. The switch expressions used in the program require Java 14 or newer. For small programs, recent JDKs can also run a single source file directly with java File.java, and jshell offers an interactive way to try out BigDecimal behavior before using it in the program.

Beyond the compiler, the ecosystem includes IDEs and editors with step-through debuggers (for example IntelliJ IDEA, Eclipse, and VS Code with the Java extensions), automatic formatters, Javadoc for generating documentation from comments, JUnit for unit testing, and the build and package managers Maven and Gradle. The project itself needs no third-party libraries: java.math (BigDecimal, RoundingMode), java.util (EnumMap, List, ArrayList, Map, Scanner) and the standard exceptions cover every requirement. The main limitation is setup overhead, since a JDK must be installed and configured and a build tool becomes worthwhile once the project has several classes and tests.

Overall, the tooling and ecosystem is more than adequate for this entire project as many libraries and functionalities are already built to satisfy similar projects.

V. Performance and Scalability

Performance isn't an issue for this use case. Interest projection runs in linear time up to the 10,000-day input cap. In testing, a maxed-out 10,000-day run finished in about two seconds, including JVM startup and compilation. Using BigDecimal is slightly slower than double, but the overhead is negligible at this scale and well worth avoiding floating-point rounding errors.

A few spots could be streamlined if memory or scale becomes a factor:

- projectInterest buffers the full table in a List before printing, whereas streaming rows directly as they compute would save RAM.
- selectAccount does an O(n)O(n) scan on an ArrayList. Swapping this for a HashMap keyed by account name would give O(1)O(1) lookups and prevent duplicate names automatically.

Thanks to JIT compilation and native concurrency support, the JVM gives us plenty of room to scale. However, the current classes aren't thread-safe, so handling concurrent users would require explicit synchronization.

VI. Comparison and Reflection

Table 1. Comparison between Java and Python (another OOP)

	Java	**Python**
Typing	Static; type errors are caught at compile time	Dynamic; type errors appear at runtime unless a checker is added
Money arithmetic	BigDecimal with explicit scale and RoundingMode; verbose method chains	decimal.Decimal supports normal operators, so formulas read closer to the specification
Code size	Longer: getters, class declarations, explicit types	Shorter: dataclasses and dicts reduce boilerplate
Error handling	Exceptions; unchecked ones such as IllegalArgumentException keep code light	Exceptions; try/except is similar in effect
Structure	Classes and encapsulation are enforced, which suits Account and ExchangeRates	Classes are optional; the program is easy to write as a plain script
Tooling	javac, JUnit, Maven/Gradle, strong IDE support; JDK setup required	Interpreter, pytest, pip; faster to start with
Performance	JIT-compiled; JVM startup cost on small programs	Slower for heavy computation; irrelevant at this scale
Concurrency	Real multithreading and concurrent collections	Threads limited by the Global Interpreter Lock (GIL) for CPU-bound work

Reflection. Java has fared to be a good match for this activity as there are a lot of things to consider when creating this banking app, mainly being the decimal rounding and conversion, along with the explicit typing and error handling. The main challenge for this programming language would be the input validation on every path and deciding how to represent foreign-currency transactions. Aside from that, because the programming language is very well-documented and abstracted, even the more complicated libraries such as BigDecimal were able to be handled. Along with that the HashMap and enums were easier to use than in other programming languages as it does not need to be built from the ground up. The problem that I faced while coding was the prohibition of adding accounts with the same name (although it is not a requirement in this MCO.) Java is easily my favorite programming language because of its wide toolset and easily accessible documentation.